

1. Crear un repositorio local

Crea un directorio con el nombre Git_P1 y posícionalte dentro del directorio creado.

```
mkdir Git_P1
cd Git_P1
```

Crea un repositorio.
git init

```
daw@daw01:~$
daw@daw01:~$ mkdir Git_P1
daw@daw01:~$ cd Git_P1
daw@daw01:~/Git_P1$ git init
```

```
daw@daw01:~/Git_P1$ git init
ayuda: Usando 'master' como el nombre de la rama inicial. Este nombre de rama pr
edeterminado
ayuda: está sujeto a cambios. Para configurar el nombre de la rama inicial para
usar en todos
ayuda: de sus nuevos repositorios, reprimiendo esta advertencia, llama a:
ayuda:
ayuda: git config --global init.defaultBranch <nombre>
ayuda:
ayuda: Los nombres comúnmente elegidos en lugar de 'master' son 'main', 'trunk'
y
ayuda: 'development'. Se puede cambiar el nombre de la rama recién creada median
te este comando:
ayuda:
ayuda: git branch -m <nombre>
Inicializado repositorio Git vacío en /home/daw/Git_P1/.git/
```

Visualiza el contenido del directorio Git_P1

ls

```
daw@daw01:~/Git_P1$ ls
daw@daw01:~/Git_P1$
```

Visualiza el contenido oculto e Git_P1 y navega con él para ver su contenido.

ls -a

```
daw@daw01:~/Git_P1$ ls -a
.  ..  .git
daw@daw01:~/Git_P1$
```

2. Comprobar el estado del repositorio local

git status para ver el estado actual de las áreas y repositorio

```
daw@daw01:~/Git_P1$ git status
En la rama master

No hay commits todavía

no hay nada para confirmar (crea/copia archivos y usa "git add" para hacerles seguimiento)
daw@daw01:~/Git_P1$
```

Crea el archivo README.md y algún archivo más.

Para ver el estado: git status

```
daw@daw01:~/Git_P1$ echo "Texto de readme" > README.md
daw@daw01:~/Git_P1$ echo "esto es una prueba" > prueba01.txt
daw@daw01:~/Git_P1$ echo "esto es otra prueba" > prueba02.txt
daw@daw01:~/Git_P1$ ls
prueba01.txt prueba02.txt README.md
daw@daw01:~/Git_P1$ git status
En la rama master

No hay commits todavía

Archivos sin seguimiento:
  (usa "git add <archivo>..." para incluirlo a lo que será confirmado)
    README.md
    prueba01.txt
    prueba02.txt

no hay nada agregado al commit pero hay archivos sin seguimiento presentes (usa "git add" para hacerles seguimiento)
daw@daw01:~/Git_P1$
```

¿En qué área o directorio, dentro de git, se encuentran los archivos que hemos creado y que aún no se han subido al repositorio?

Se encuentran en el directorio de trabajo, como archivos sin seguimiento

3. Añadir cambios al área de espera/preparación

git add

git status para comprobar

```
git add para nuevos documentos
daw@daw01:~/Git_P1$ git add README.md
daw@daw01:~/Git_P1$ git status
En la rama master

No hay commits todavía

Cambios a ser confirmados:
  (usa "git rm --cached <archivo>..." para sacar del área de stage)
nuevos archivos: README.md

Archivos sin seguimiento:
  (usa "git add <archivo>..." para incluirlo a lo que será confirmado)
prueba01.txt
prueba02.txt

daw@daw01:~/Git_P1$
```

¿Qué tendremos ahora en el área de trabajo? ¿En qué área está el documento README.md?

En el área de trabajo quedan prueba01.txt y prueba02.txt , que siguen sin seguimiento. El documento readme está en el área de espera, listo para hacer commit.

Modifica el archivo README.md y comprueba

```
daw@daw01:~/Git_P1$ echo "Esto es otra línea en readme... OJO que es con >>" >> README.md
daw@daw01:~/Git_P1$ git status
En la rama master

No hay commits todavía

Cambios a ser confirmados:
  (usa "git rm --cached <archivo>..." para sacar del área de stage)
nuevos archivos: README.md

Cambios no rastreados para el commit:
  (usa "git add <archivo>..." para actualizar lo que será confirmado)
  (usa "git restore <archivo>..." para descartar los cambios en el directorio de trabajo)
modificados:      README.md

Archivos sin seguimiento:
  (usa "git add <archivo>..." para incluirlo a lo que será confirmado)
prueba01.txt
prueba02.txt

daw@daw01:~/Git_P1$
```

¿Qué sucede si volvemos a modificar el documento README.md? ¿En qué área estaría?

La versión anterior sigue en el área de espera y la versión nueva está en el directorio de trabajo. Git status lo muestra en las dos secciones hasta que hagamos otro git add

4. Añadir cambios al área de espera/preparación

git commit y comprueba con git log

```
daw@daw01:~/Git_P1$ git commit -m "este es mi primer commit"
[master (commit-raíz) b4d08dc] este es mi primer commit
1 file changed, 1 insertion(+)
create mode 100644 README.md
daw@daw01:~/Git_P1$ git status
En la rama master
Cambios no rastreados para el commit:
  (usa "git add <archivo>..." para actualizar lo que será confirmado)
  (usa "git restore <archivo>..." para descartar los cambios en el directorio de trabajo)
      modificados:   README.md

Archivos sin seguimiento:
  (usa "git add <archivo>..." para incluirlo a lo que será confirmado)
      prueba01.txt
      prueba02.txt

sin cambios agregados al commit (usa "git add" y/o "git commit -a")
daw@daw01:~/Git_P1$ git log
commit b4d08dc11a6fdc69b8b8dd9a16f790ce6ee9abb7 (HEAD -> master)
Author: Tello <Telly1@gmail.com>
Date:   Wed Sep 16 14:19:57 2026 +0200

    este es mi primer commit
```

¿Por qué ya no aparece ningún fichero?

Porque el commit ha movido todos los cambios al repositorio y el directorio de trabajo coincide con el último commit

¿Qué significa cada parte de la respuesta de git log?

commit + código: identificador único (hash) del commit.

(HEAD -> master): rama en la que estamos y commit actual.

Author: quién hizo el commit

Date: fecha y hora del commit

El texto final es el mensaje del commit.

5. Ignorar ficheros de nuestro directorio de trabajo

Crea el fichero .gitignore en la raíz del directorio de trabajo. Incluye que ignore los archivos con extensión .tmp. Puedes hacerlo con cualquier editor de texto o directamente desde la línea de comandos.

```
daw@daw01:~/Git_P1$ echo "*.tmp" > .gitignore
daw@daw01:~/Git_P1$ git add . gitignore
fatal: ruta especificada 'gitignore' no concordó con ningún archivo
daw@daw01:~/Git_P1$ git add .gitignore
daw@daw01:~/Git_P1$ git commit -m "actualización de .gitignore"
Orden «commit» no encontrada. Quizá quiso decir:
  la orden «commix» del paquete snap «commix (v4.0-2-g79909148)»
Consulte «snap info <nombre del snap>» para ver más versiones.
daw@daw01:~/Git_P1$ git commit -m "actualización de .gitignore"
[master 049fa4f] actualización de .gitignore
1 file changed, 1 insertion(+)
create mode 100644 .gitignore
```

Log:

```
daw@daw01:~/Git_P1$ git log
commit 049fa4fff260d0ac2c48fb1422e49cdced80a5e4 (HEAD -> master)
Author: Tello <TelloTy1@gmail.com>
Date:   Wed Sep 16 14:24:47 2026 +0200

    actualización de .gitignore

commit b4d08dc11a6fdc69b8b8dd9a16f790ce6ee9abb7
Author: Tello <TelloTy1@gmail.com>
Date:   Wed Sep 16 14:19:57 2026 +0200

    este es mi primer commit
```

Crea el archivo texto.txt.

```
daw@daw01:~/Git_P1$ echo "texto del archivo temporal 1" > temporal_1.tmp
daw@daw01:~/Git_P1$ echo "texto del archivo temporal 2" > temporal_2.tmp
daw@daw01:~/Git_P1$ echo "texto del archivo temporal 3" > temporal_3.tmp
daw@daw01:~/Git_P1$ echo "texto el archivo texto" > texto.txt
daw@daw01:~/Git_P1$
```

Añade todos los archivos que han sido modificados en el directorio de trabajo al área de espera/preparación y revisa el status

```
daw@daw01:~/Git_P1$ git add .
daw@daw01:~/Git_P1$ git status
En la rama master
Cambios a ser confirmados:
  (usa "git restore --staged <archivo>..." para sacar del área de stage)
    modificados:   README.md
    nuevos archivos: prueba01.txt
    nuevos archivos: prueba02.txt
    nuevos archivos: texto.txt

daw@daw01:~/Git_P1$
```

¿Por qué no aparecen los archivos .tmp? ¿En qué situaciones podría interesarnos el no incluir algún archivo o directorio en el repositorio?

No aparecen porque .gitignore contiene *.tmp y Git ignora esos archivos. Interesa ignorar archivos temporales, logs, configuración local, contraseñas o claves

6 Comprobar diferencias.

Git diff para mostrar los cambios en archivos que aun no se han enviado al área de espera.

```
daw@daw01:~/Git_P1$ git diff
daw@daw01:~/Git_P1$
```

Get status para comprobar modificaciones.

```
daw@daw01:~/Git_P1$ git status
En la rama master
Cambios a ser confirmados:
  (usa "git restore --staged <archivo>..." para sacar del área de stage)
    modificados:   README.md
    nuevos archivos: prueba01.txt
    nuevos archivos: prueba02.txt
    nuevos archivos: texto.txt

daw@daw01:~/Git_P1$
```

¿Por qué no aparecen nada si hemos creado un fichero nuevo "texto.txt"?

Porque git diff solo muestra los cambios que aún no están en el área de espera. texto.txt ya se añadió con git add . , así que está en el área de espera

Añade una nueva línea de texto en el fichero “prueba01.txt” y git status para ver el estado actual.

```
daw@daw01:~/Git_P1$ echo "esto es una nueva linea" >> prueba01.txt
daw@daw01:~/Git_P1$ git status
En la rama master
Cambios a ser confirmados:
  (usa "git restore --staged <archivo>..." para sacar del área de stage)
    modificados:   README.md
    nuevos archivos: prueba01.txt
    nuevos archivos: prueba02.txt
    nuevos archivos: texto.txt

Cambios no rastreados para el commit:
  (usa "git add <archivo>..." para actualizar lo que será confirmado)
  (usa "git restore <archivo>..." para descartar los cambios en el directorio de
trabajo)
    modificados:   prueba01.txt

daw@daw01:~/Git_P1$
```

Identifica qué ficheros están modificados en el directorio de trabajo y en área de espera/preparación.

prueba01.txt está modificado en el directorio de trabajo (en rojo) y texto.txt está en el área de espera (en verde).

Git diff para comprobar de nueva las modificaciones de los documentos que tenemos en el repositorio en comparación con esos mismos documentos en el directorio del trabajo

```
daw@daw01:~/Git_P1$ git diff
diff --git a/prueba01.txt b/prueba01.txt
index 97ab2fe..4aac4a5 100644
--- a/prueba01.txt
+++ b/prueba01.txt
@@ -1,2 @@
  esto es una prueba
+esto es una nueva linea
daw@daw01:~/Git_P1$
```

Utilizando un editor de texto, elimina la primera línea del documento “prueba01.txt” y crea una nueva línea con el contenido “esto es otra nueva línea”

```
GNU nano 7.2 prueba01.txt *
esto es otra nueva línea
esto es una nueva línea
```

git diff para comprobar.

```
daw@daw01:~/Git_P1$ git diff
diff --git a/prueba01.txt b/prueba01.txt
index 97ab2fe..8982f0c 100644
--- a/prueba01.txt
+++ b/prueba01.txt
@@ -1,2 @@
-esto es una prueba
+esto es otra nueva línea
+esto es una nueva línea
daw@daw01:~/Git_P1$
```

¿Cómo se identifica el contenido nuevo que se ha incluido y el que se ha borrado?

Las líneas que empiezan por + (verde) son contenido nuevo y las que empiezan por - (rojo) son contenido borrado.

Git log para comprobar el estado

```
daw@daw01:~/Git_P1$ git log
commit 049fa4ffff260d0ac2c48fb1422e49cdced80a5e4 (HEAD -> master)
Author: Tello <Telly1@gmail.com>
Date:   Wed Sep 16 14:24:47 2026 +0200

    actualización de .gitignore

commit b4d08dc11a6fdc69b8b8dd9a16f790ce6ee9abb7
Author: Tello <Telly1@gmail.com>
Date:   Wed Sep 16 14:19:57 2026 +0200

    este es mi primer commit
daw@daw01:~/Git_P1$
```

comprobar con git diff el commit b4.... Que sale

```
daw@daw01:~/Git_P1$ git diff b4d08dc11a6fdc69b8b8dd9a16f790ce6ee9abb7
diff --git a/.gitignore b/.gitignore
new file mode 100644
index 0000000..1944fd6
--- /dev/null
+++ b/.gitignore
@@ -0,0 +1 @@
+*.tmp
diff --git a/README.md b/README.md
index 9191cd4..4116b94 100644
--- a/README.md
+++ b/README.md
@@ -1,2 @@
-Texto de readme
+Esto es otra línea en readme... OJO que es con >>
diff --git a/prueba01.txt b/prueba01.txt
new file mode 100644
index 0000000..8982f0c
--- /dev/null
+++ b/prueba01.txt
@@ -0,0 +1,2 @@
```

Comprueba las modificaciones realizadas con git show

```
commit 049fa4fff260d0ac2c48fb1422e49cdced80a5e4 (HEAD -> master)
Author: Tello <Telly1@gmail.com>
Date:   Wed Sep 16 14:24:47 2026 +0200

    actualización de .gitignore

diff --git a/.gitignore b/.gitignore
new file mode 100644
index 0000000..1944fd6
--- /dev/null
+++ b/.gitignore
@@ -0,0 +1 @@
+*.tmp
daw@daw01:~/Git_P1$
```

¿Qué información diferente nos muestra git show con git log y git diff?

git log solo muestra los datos del commit y git diff solo muestra los cambios. git show muestra las dos cosas: los datos del commit y los cambios que introdujo.